

Trabajo Fin de Grado

TRUQU: TRuffle QUality Recognition

TRUQU: Reconocimiento de Calidad de Trufas

Autor

D. Diego Hernández Cortés

Directores

Dra. D^a Piedad Garrido Picazo

Dr. D Julio A. Sangüesa Escorihuela

Escuela Universitaria Politécnica de Teruel

2023



Tabla de Contenidos

INTRODUCCIÓN, MOTIVACIÓN Y OBJETIVOS.....	- 1 -
CONTEXTUALIZACIÓN	- 3 -
ESTADO DEL ARTE	- 6 -
PROTOTIPO DE RED NEURONAL CONVOLUCIONAL	- 9 -
ENTORNO DE TRABAJO	- 9 -
PREPROCESAMIENTO DE DATOS	- 12 -
DESARROLLO DE LA RED NEURONAL.....	- 14 -
CREACIÓN DEL DATASET	- 20 -
PRUEBAS REALIZADAS.....	- 25 -
<i>Cloud Vision AI</i>	- 25 -
<i>Hiperparámetros</i>	- 28 -
<i>Pruebas utilizando GPU</i>	- 29 -
<i>Pruebas utilizando CPU</i>	- 30 -
<i>Comparativa entre CPU y GPU</i>	- 31 -
<i>Modelo final</i>	- 33 -
PROBLEMAS ENCONTRADOS	- 39 -
LICENCIA SOFTWARE Y DOCUMENTAL	- 40 -
CONCLUSIONES Y TRABAJO FUTURO	- 41 -
REFERENCIAS BIBLIOGRÁFICAS	- 42 -

Índice de figuras

Figura 1. Perceptron	- 3 -
Figura 2. Perceptron multicapa	- 4 -
Figura 3. Red neuronal convolucional	- 5 -
Figura 4. Máquina utilizada para el entrenamiento de la red	- 10 -
Figura 5. Función de rotación de imagen	- 12 -
Figura 6. Creación de los conjuntos de datos	- 13 -
Figura 7. Red neuronal de partida	- 14 -
Figura 8. Carga de imágenes	- 15 -
Figura 9. Preparación de las imágenes	- 16 -
Figura 10. Cambio de etiquetas a one-hot	- 17 -
Figura 11. Ejemplo de one-hot	- 17 -
Figura 12. Funciones de validación	- 18 -
Figura 13. Configuración de la red neuronal	- 19 -
Figura 14. Carga de imágenes	- 21 -
Figura 15. Configuración de fuente externa	- 22 -
Figura 16. Creación del bucket	- 22 -
Figura 17. Código para dar permiso de conexión	- 23 -
Figura 18. Consulta SQL para crear conexión entre Bucket y BigQuery	- 23 -
Figura 19. Consulta SQL para obtener una imagen	- 23 -
Figura 20. Resultado de la consulta	- 24 -
Figura 21. Comparativa usando Vision AI	- 26 -
Figura 22. Comparativa usando Vision AI	- 27 -
Figura 23. Pruebas con GPU	- 29 -
Figura 24. Pruebas con CPU	- 30 -
Figura 25. Comparativa de entornos	- 31 -
Figura 26. Gráfica del entrenamiento con lr fijo	- 33 -
Figura 27. Learning rate variable	- 34 -
Figura 28. Gráfica del entrenamiento con lr variable	- 35 -
Figura 29. Comparativa de modelos	- 35 -
Figura 30. Código para probar modelo entrenado	- 36 -
Figura 31. Predicción con lr fijo	- 37 -
Figura 32. Predicción modelo con lr variable	- 38 -
Figura 33. Logotipo de licencia BSD	- 40 -

Índice de tablas

Tabla 1. Resumen de los artículos utilizados en el estado del arte.....- 8 -

Tabla 2. Tabla resumen del dataset.....- 13 -

Tabla 3. Comparativa de tiempos de las pruebas.....- 32 -

Resumen

Este Trabajo de Fin de Grado (TFG) se centra en la aplicación de técnicas avanzadas de Inteligencia Artificial (IA) para el reconocimiento y clasificación de trufas como primera medida de su calidad.

En una primera fase, se han recopilado imágenes representativas de trufas de distintas especies para crear el dataset utilizado para entrenar y evaluar la red neuronal.

Posteriormente, se ha implementado una red neuronal convolucional utilizando las bibliotecas TensorFlow y Keras. La elección de este enfoque se basó en la capacidad demostrada por estas redes para extraer características relevantes de imágenes, lo que resulta especialmente útil en tareas de clasificación de objetos.

La red neuronal, con el dataset creado, ha demostrado habilidades significativas en el reconocimiento preciso y la clasificación efectiva de trufas haciendo uso de tecnologías como TensorFlow, Keras, Google BigQuery, Google Cloud Vision, ...

Este TFG no solo contribuye al campo de la IA, sino que también presenta una aplicación práctica en la industria agroalimentaria, donde el reconocimiento preciso de trufas puede tener impactos significativos en la calidad y comercialización de estos productos.

Palabras clave

Reconocimiento de trufa, red neuronal convolucional, Tensorflow, GPU, inteligencia artificial

Abstract

This Final Degree Project focuses on the application of advanced Artificial Intelligence (AI) techniques for the recognition and classification of truffles as an initial measure of their quality.

In the first phase, representative images of truffles from different species were collected to create the dataset used for train and evaluate the neural network.

Subsequently, a convolutional neural network was implemented using TensorFlow and Keras libraries. The choice of this approach was based on the demonstrated ability of these networks to extract relevant features from images, which is especially useful in object classification tasks.

The neural network, with the created dataset, demonstrated significant skills in accurate recognition and effective classification of truffles, using technologies such as TensorFlow, Keras, Google BigQuery, Google Cloud Vision, and others.

This project not only contributes to the field of AI but also presents a practical application in the agri-food industry, where precise Truffle recognition can have significant impacts on the quality and marketing of these products.

Keywords

Truffle recognition, convolutional neural network, TensorFlow, GPU, artificial intelligence

Introducción, motivación y objetivos

La Inteligencia Artificial (IA) es cada vez más conocida y utilizada en la actualidad, los avances de la IA están haciendo que este tema esté en boca de todos desde hace ya varios años. Cada vez aparecen más técnicas de IA diferentes, que, aplicadas de manera correcta, son capaces de llevar a cabo infinidad de aplicaciones en distintos sectores.

Una de las disciplinas de la IA es el aprendizaje automático (en inglés Machine Learning). Este término se utiliza para describir la creación y desarrollo de algoritmos que tienen la capacidad finita de aprendizaje independientemente de cuantos datos adquieran. Estos algoritmos son entrenados utilizando información para identificar patrones y relaciones en los datos. Estando presente en una amplia variedad de sectores y contando con aplicaciones que van desde la realización de diagnósticos médicos, a la conducción autónoma de vehículos, predicciones en el mercado bursátil, etc.

Dentro del aprendizaje automático, existe una rama conocida como el aprendizaje profundo (en inglés Deep Learning) que se centra en la creación y entrenamiento de redes neuronales profundas. Estas redes neuronales profundas constan de una gran cantidad de capas, lo que permite que capturen y aprendan a partir de conjuntos de datos cada vez más complejos, abstractos y grandes. Esto les da la capacidad de mejorar su rendimiento y obtener un alto grado de precisión en una amplia variedad de dominios como el reconocimiento de imágenes, detección de plagas, enfermedades, reconocimiento de voz, etc.

Es en materia de reconocimiento de imágenes donde se centra, a nivel tecnológico, la temática de este Trabajo Fin de Grado (TFG), ya que el primer parámetro para el reconocimiento de la calidad de una trufa es conseguir que un agente software la clasifique en una de las más de setenta especies que existen alrededor del mundo.

En concreto este TFG se va a centrar en la clasificación de dieciocho tipos de trufas comunes en Europa mediante el uso de técnicas de aprendizaje automático. Por un lado, para comenzar con el proceso de detección de calidad de una trufa, la clasificación de la trufa se va a realizar teniendo en cuenta tanto la parte interior como la parte exterior de la trufa.

Aunque existen numerosas aplicaciones y proyectos relacionados con el reconocimiento y la clasificación de distintos tipos de hongos, como las setas, no ocurre lo mismo con las trufas, por lo que se decidió abordar este desafío dada su importancia en la provincia de Teruel. La especie 'Tuber Melanosporum', considerada el diamante negro de la gastronomía, ya ocupa alrededor de 10.000 hectáreas de producción en la provincia, Sarrión destaca como el epicentro mundial de la producción de esta especie, abasteciendo gran parte de la demanda global. La trufa de Teruel, una marca registrada desde 2005, ha recibido la designación de Indicación Geográfica Protegida (IGP) debido a sus características únicas y a la tradición de cultivo de trufas en la región. La IGP es una forma de reconocer y proteger la calidad y autenticidad de productos específicos originarios de una región geográfica determinada.

Por otro lado, las redes neuronales convolucionales son especialmente eficientes para este tipo de tareas, gracias a su capacidad para aprender y representar características visuales de manera jerárquica y automática. Esta es la razón por la que este tipo de redes neuronales han sido las elegidas para llevar a cabo este TFG, cuyos objetivos son:

- Creación de un dataset con imágenes de los distintos tipos de trufas seleccionados.
- Trabajar el parámetro de calidad, dando el primer paso con la creación de un software de reconocimiento y clasificación de la trufa mediante la construcción de una RNA (Red Neuronal Artificial)
- Realización de pruebas, de la RNA creada, en distintos entornos de desarrollo y trabajando con Unidad de Procesamiento Central (CPU) y Unidad de Procesamiento Gráfico (GPU).

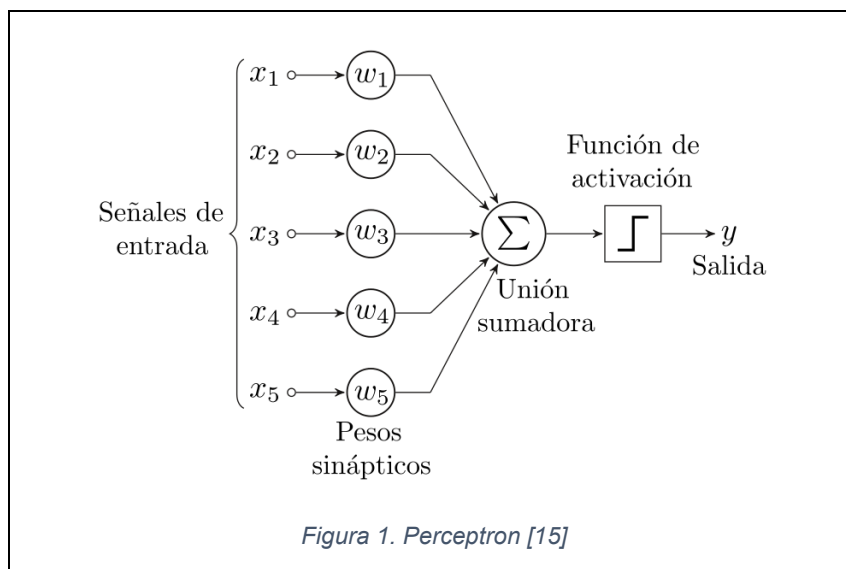
Este TFG comienza con una introducción, donde se hace un repaso a la historia de las redes neuronales y se cuenta su funcionamiento. Seguidamente se tratan los objetivos principales de este trabajo y se realiza un estudio del estado del arte, tratando tanto las tecnologías utilizadas como las aplicaciones actuales que se asemejan a este trabajo. Posteriormente se presenta la propuesta donde se lleva a cabo la implementación de una Red Neuronal Convolucional utilizando el entorno de TensorFlow y se exponen los resultados obtenidos. Para finalizar, se presentan unas conclusiones y el trabajo futuro.

Contextualización

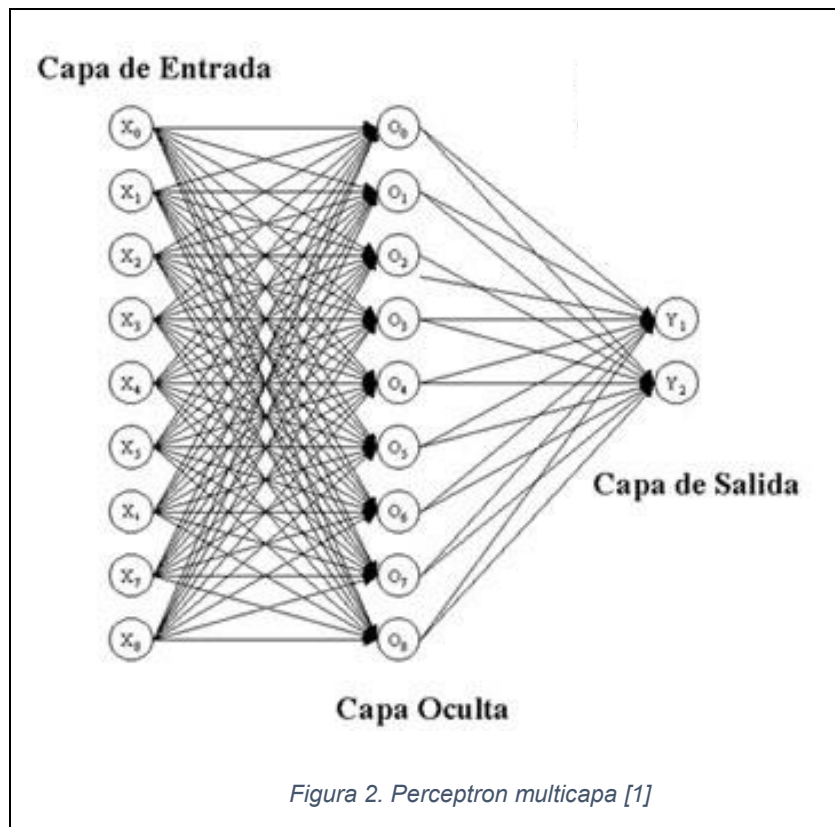
Para comenzar, es necesario conocer como han ido evolucionando las redes neuronales con el paso de los años [13] [11].

El primer paso en cuanto a redes neuronales se refiere lo dio Frank Rosenblatt [6] en la década de **1960**, quien influenciado por los trabajos de Walter Pitts y Warren McCulloch, desarrolló el perceptrón (Figura 1), creación que dio lugar al nacimiento de las redes neuronales de hoy en día.

El perceptrón, con un funcionamiento similar al de una neurona [14], opera mediante la toma de múltiples entradas binarias, denotadas como x_1 , x_2 , etc., para generar una única salida binaria. Rosenblatt introdujo el concepto de "pesos", representados por w_1 , w_2 , etc., que son valores reales que indican la importancia de cada entrada en relación con la salida. La salida de la neurona se determina por la comparación de la suma de las multiplicaciones de los pesos por las entradas con un umbral específico, resultando en una salida de 1 o 0.



Posteriormente, a finales de la década de **1960**, el perceptron evolucionó creando modelos multicapa compuestos de perceptrones. A diferencia de los perceptrones de una sola capa, los **perceptrones multicapa** constan de al menos tres capas de nodos: una capa de entrada, una o varias capas ocultas y una capa de salida. Cada nodo en una capa está conectado a todos los nodos de la capa siguiente, y cada conexión tiene un peso asociado. Permitiendo así solucionar problemas más complejos (Figura 2).

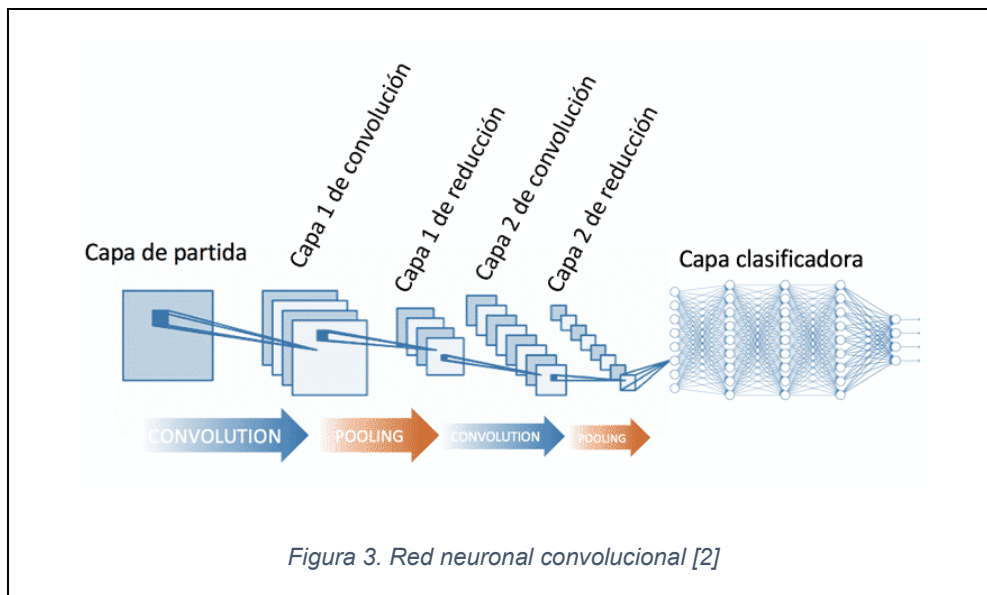


En la década de **1980**, y para conseguir que las redes de neuronas aprendieran por sí solas se introdujo un nuevo tipo de neuronas, las **neuronas sigmoides**. Las neuronas sigmoides funcionan de manera parecida al perceptron, pero permiten que las entradas, en vez de ser ceros o unos, puedan tener valores reales como 0,6 o 0,422. Con este avance, los valores de salida dejaban de ser binarios. Las neuronas sigmoides son la primera función de activación.

Gracias al algoritmo de **retropropagación** (en inglés, backpropagation [19]) se hizo posible el entrenamiento de redes neuronales de múltiples capas. Este algoritmo funciona ajustando los pesos de la red en función del error calculado entre la salida predicha y la salida deseada. Este proceso se repite iterativamente hasta que la red aprende.

En **1989**, Yann LeCun, Léon Bottou, Yoshua Bengio y Patrick Haffner presentaron un artículo titulado "Object Recognition with Gradient-Based Learning". En este trabajo, propusieron una arquitectura de **red neuronal convolucional** y utilizaron el algoritmo de retropropagación para entrenarla. Aunque esta implementación era más simple en comparación con las redes neuronales convolucionales modernas, sentó las bases para el desarrollo posterior de estas redes.

Esta arquitectura constaba de varias capas encargadas de realizar extracciones de información y clasificación de características, se llevaba a cabo una segmentación de la imagen en distintos campos que proporcionaban información a una capa convolucional encargada de extraer características clave de la imagen original. Posteriormente, se aplicaba un proceso de agrupación (pooling) que reducía la dimensionalidad de las características extraídas, preservando la información más relevante. Se repetía este ciclo con una nueva convolución y otra fase de agrupación, alimentando así una red neuronal multicapa. La salida final de la red consistía en un conjunto de nodos destinados a la clasificación, por ejemplo, un nodo asignado a cada número del 0 al 9, es decir, 10 nodos que se activaban individualmente para indicar la clasificación correspondiente (Figura 3).



En resumen, la contextualización de este TFG concluye con la introducción de las redes neuronales convolucionales, las cuales constituyen la base utilizada en el desarrollo de este proyecto. A partir de las Redes Neuronales Convolucionales, han surgido nuevos tipos de redes que han contribuido significativamente al campo de la inteligencia artificial. Entre estas, destacan las Redes Bayesianas Profundas (DBN), Redes Generativas Adversarias (GAN) y las Redes Neuronales Residuales (ResNet), que cuentan con tecnologías más avanzadas que no se han utilizado en este proyecto.

Estado del arte

Este TFG se centra en el reconocimiento de trufas, un hongo que se encuentra debajo de la tierra. Existen distintos proyectos y estudios acerca del reconocimiento y la clasificación de distintos tipos de hongos.

En concreto, se han seleccionado cuatro iniciativas publicadas en dos congresos y una revista, más un proyecto vigente a nivel nacional.

El Colegio de ingenieros de Sri Sivasubramaniva Nadar de India ha publicado un artículo [16] en el que se detalla un enfoque basado en el aprendizaje profundo para la obtención y extracción de características para la clasificación de especies de hongos, más concretamente, setas. Su flujo de trabajo aprovecha arquitecturas como ResNeXt y EfficientNet, las entrena mediante transferencia de aprendizaje en un conjunto de datos de imágenes de setas para la extracción de características.

Otro proyecto relacionado con el reconocimiento y clasificación de hongos es el realizado en la Universidad de Copenhague [20]. Se trata de un trabajo de reconocimiento de imágenes con aproximadamente 1394 especies de hongos en el que se han utilizado redes neuronales convolucionales profundas. Esta App permite a los usuarios identificar automáticamente los especímenes observados, y a su vez, muestra información valiosa acerca de cada tipo de hongo encontrado. Este proyecto cuenta con una gran usabilidad para los usuarios, ya que permite identificar hongos y obtener información acerca de cada especie de manera rápida y desde un dispositivo móvil. Cabe destacar que dentro del conjunto de especies de hongos con las que trabaja este sistema, no se encuentran las trufas, si no que está más enfocado al reconocimiento de setas.

Otra aplicación con el propósito de reconocer hongos es la creada en colaboración por la Universidad West Bohemia en la República Checa, Universidad de Praga y por la Universidad de Copenhague [5]. La aplicación, denominada FungiVision, cuenta con una interfaz simple para un uso. Utiliza un conjunto de datos creado por la Universidad de Copenhague [17] compuesto por imágenes de distintos tipos de hongos, centrándose en las setas, utilizando la arquitectura Vision Transformer (ViT) consigue cerca de un 93% de acierto.

En el ámbito de las trufas en concreto, existe un proyecto vigente que se centra en el análisis de imágenes en relación con las trufas. Este proyecto, denominado “SMARTRUFFLE”, es un proyecto de análisis de imágenes y datos para el cultivo de trufas. Es llevado a cabo por AERTIC (Agrupación Empresarial Innovadora del Sector TIC de La Rioja), junto a sus empresas socias Panoimagen y SpectralGeo. El objetivo de SMARTRUFFLE es el desarrollar un sistema que ayude a los agricultores del cultivo de la trufa a encontrar y clasificar trufas de alta calidad. Para lograrlo, se ha desarrollado una aplicación Web GIS (en inglés, Geographic Information System) que utiliza algoritmos de inteligencia artificial para buscar patrones que indiquen la presencia de trufas y clasificarlas según el momento óptimo para su recolección.

Como conclusión sobre este estado del arte que aborda estudios desde el año 2018, tal y como se puede observar en la Tabla 1, existen aplicaciones y proyectos relacionados con el reconocimiento de algunas clases de hongos [16] [20] [5] [17], todos ellos setas u hongos que no se crían en el interior de la tierra. Por lo tanto, no existe en la actualidad ningún tipo de desarrollo como el propuesto, que no sólo comience trabajando la calidad de la trufa consiguiendo clasificarla en una de las 18 especies más comunes en Europa, sino que analice dicha clasificación desde distintos entornos de desarrollo involucrando en su rendimiento al hardware, es decir, trabajar tanto con CPU como GPU.

Artículo	Fuente	Tecnología	Año
Classification of Fungi Species: A Deep Learning Based Image Feature Extraction and Gradient Boosting Ensemble Approach [5]	CEUR-Workshop	ResNeXt y EfficientNet	2022
Fungi Recognition: A Practical Use Case [20]	IEEE Conference on Computer Vision and Pattern Recognition (CVPR)	Redes neuronales convolucionales	2018
Danish Fungi 2020 – Not Just Another Image Recognition Dataset [17]	IEEE Conference on Computer Vision and Pattern Recognition (CVPR)	Redes neuronales convolucionales y Vision Transformer (ViT)	2020
Automatic Fungi Recognition: Deep Learning Meets Mycology [16]	Sensor(Publicado por MDPI)	Vision Transformer (ViT)	2022

Tabla 1. Resumen de los artículos utilizados en el estado del arte

Prototipo de red neuronal convolucional

La propuesta que se presenta en este TFG no solo se centra en la creación de una red neuronal convolucional que sea capaz de reconocer y clasificar entre dieciocho tipos de trufas, sino que abarca también la importancia de la configuración del entorno de trabajo que se va a utilizar para llevar a cabo su desarrollo.

Entorno de trabajo

En este apartado, se va a llevar a cabo una descripción de la máquina utilizada para poner en marcha la red neuronal, también se describirá el entorno de trabajo utilizado tanto para el desarrollo como para su puesta en marcha.

En primer lugar, la máquina utilizada cuenta con un procesador Intel Core i9-13900k con 24 núcleos. Esto permite que la máquina pueda realizar varias tareas de manera simultánea. Dispone de una tarjeta gráfica GeForce RTX 3070 Ti GAMING X TRIO 8G, de alto rendimiento fabricada por MSI y que forma parte de la serie GeForce de Nvidia. La máquina tiene 64 GB de RAM dividida en 4 módulos de memoria de 16 GB DDR5 HYPERX.

Por último, el sistema operativo elegido ha sido Ubuntu 22.04. LTS, ya que proporciona una plataforma estable y sencilla de usar para la creación y desarrollo de aplicaciones de IA. Este sistema operativo cuenta con una gran cantidad de herramientas compatibles con el software que se va a utilizar.

En el contexto de este TFG, se ha optado por aprovechar la Unidad de Procesamiento Gráfico, conocida como GPU (Graphics Processing Unit en inglés), para llevar a cabo el entrenamiento de redes neuronales.

Esto se debe a que el uso de una tarjeta gráfica de alta calidad conlleva resultados significativamente superiores en comparación con el uso de una Unidad Central de Procesamiento (CPU) en tareas de entrenamiento.

Por lo tanto, la GPU se utiliza principalmente para realizar cálculos intensivos de manera paralela en el procesamiento de los datos de entrenamiento de las redes neuronales.

La memoria RAM desempeña un papel importante al garantizar que la GPU cuente con suficiente información de entrenamiento para operar de manera eficiente y sin interrupciones.

Además, la CPU es la encargada de la gestión general del sistema y la transferencia de datos entre la memoria RAM y la GPU. Esta configuración se busca para maximizar la eficiencia de la GPU durante el proceso de entrenamiento.



Figura 4. Máquina utilizada para el entrenamiento de la red.

Una vez vistas las especificaciones principales de la máquina que se va a utilizar para el entrenamiento de la red neuronal, se va a detallar la librería utilizada para la creación de la Red Neuronal Convolucional, así como el proceso de instalación de esta librería.

El lenguaje de programación elegido ha sido Python ya que cuenta con librerías interesantes para la creación de redes neuronales. Se ha utilizado **TensorFlow** [21] ya que permite utilizar tanto la CPU como la GPU para la realización de los entrenamientos de la red neuronal. Esto permite sacar el máximo partido a la máquina utilizada y da opción a poder realizar distintas comparativas entre redes neuronales entrenadas mediante CPU y redes neuronales entrenadas mediante GPU.

TensorFlow es un marco de trabajo de aprendizaje automático que permite a los desarrolladores construir, entrenar y desplegar modelos de redes neuronales y otros algoritmos de aprendizaje automático de manera eficiente. Diseñado por Google, se ha convertido en una de las herramientas más utilizadas en el ámbito del aprendizaje automático.

TensorFlow utiliza de manera muy eficiente las unidades de procesamiento gráfico (GPU) y las unidades de procesamiento tensorial (TPU) para así conseguir acelerar el entrenamiento de las redes neuronales, aprovechando al máximo el poder de cómputo de la máquina en la que se está utilizando.

También proporciona capas de abstracción que facilitan las tareas de creación de redes neuronales, esto permite crear redes neuronales de manera mucho más sencilla y rápida. Esto hace de TensorFlow una herramienta flexible y escalable.

Una vez seleccionado TensorFlow como entorno de trabajo. Se ha llevado a cabo la instalación de esta librería en la máquina. Para ello, se han seguido los siguientes pasos:

- Se ha confirmado que se satisfacen los requisitos del sistema, los cuales son los siguientes:
 - Ubuntu 16.04 o superior
 - Python 3.6 o superior
 - Tarjeta gráfica Nvidia compatible con CUDA
- Se ha descargado el entorno de Miniconda, el cual lleva integrado TensorFlow.
 - `curl https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o Miniconda3-latest-Linux-x86_64.sh`
 - `bash Miniconda3-latest-Linux-x86_64.sh`
- Una vez descargado, se ha creado un entorno de trabajo y se ha activado.
 - `conda create --name tf python=3.9.18`
 - `conda activate tf`
- Una vez creado el entorno de trabajo, se ha instalado TensorFlow, especificando también la instalación de los drivers de Cuda, necesarios para poder usar la GPU a la hora de realizar entrenamientos.
 - `python3 -m pip install tensorflow[and-cuda]`
- Por último, se ha comprobado la instalación haciendo uso de la siguiente línea de código:
 - `python3 -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"`

Preprocesamiento de datos

En cuanto al conjunto de imágenes utilizado, y dado que no se ha encontrado ningún dataset con imágenes de trufas, se han recopilado imágenes de 18 tipos de trufas comunes en Europa y se han conseguido alrededor de 170 imágenes útiles para el entrenamiento de la red neuronal convolucional.

Estas imágenes, se han almacenado en directorios dependiendo del tipo de trufa, por lo que el dataset contendrá los distintos directorios correspondientes a los tipos de trufas seleccionados.

Una vez recopiladas las imágenes, y como no se han encontrado las imágenes suficientes como para realizar un entrenamiento, se han tenido que aplicar métodos de **rotación** de imágenes para conseguir un conjunto de datos más extenso, se han realizado 5 rotaciones aleatorias de entre 40 y 360 grados a todas las imágenes del dataset. A continuación, se muestra el código de cómo se ha realizado la rotación (Figura 5). Se puede consultar todo el código en el repositorio de GitHub [24] creado para este TFG.

```
#Rota una imagen con un ángulo aleatorio entre 40º y 360º
def rotaImagen(imagen):
    angulo = rn.randint(40,360)
    try:
        img_rotada = imutils.rotate(imagen,angulo)
    except Exception as e:
        print('Error al rotar la imagen')
        img_rotada = 0
    return img_rotada
```

Figura 5. Función de rotación de imagen

También se ha invertido cada imagen en modo **espejo** para obtener un mayor número de imágenes. Una vez realizadas las 5 rotaciones y el modo espejo, se han conseguido obtener un total de 1077 imágenes. Estas 1077 imágenes se han dividido en imágenes de entrenamiento e imágenes de test, la división se puede ver en la Tabla 2.

CONJUNTO	NÚMERO DE IMÁGENES
Entrenamiento	1042
Test	35

Tabla 2. Tabla resumen del dataset

Como en este TFG se busca clasificar las imágenes en distintas categorías, cada imagen deberá tener asignada una etiqueta con el tipo de trufa correspondiente. Para asignar esta etiqueta, se obtendrá el nombre del directorio al que pertenece cada imagen y se le asignará el nombre de éste a la etiqueta de cada imagen, de esta manera cada imagen estará clasificada en función de su categoría.

Una vez asignadas las categorías a cada imagen, se ha realizado la división de las imágenes al conjunto de entrenamiento y al conjunto de test. Para llevar a cabo la división, se ha creado la siguiente función que mediante el uso del método `train_test_split()` proveniente de la librería `sklearn`, divide los datos de en función del parámetro `test_size`, posteriormente se han convertido los conjuntos de train y test al formato float y finalmente se normalizan los conjuntos dividiendo entre 255 como se observa en la Figura 6.

```
def creaConjuntosDatos(X,Y):
    train_X,test_X,train_Y,test_Y = train_test_split(X,Y,test_size=TEST_SIZE)

    train_X = train_X.astype('float32')
    test_X = test_X.astype('float32')
    train_X = train_X / 255.
    test_X = test_X / 255.

    return train_X,test_X,train_Y,test_Y
```

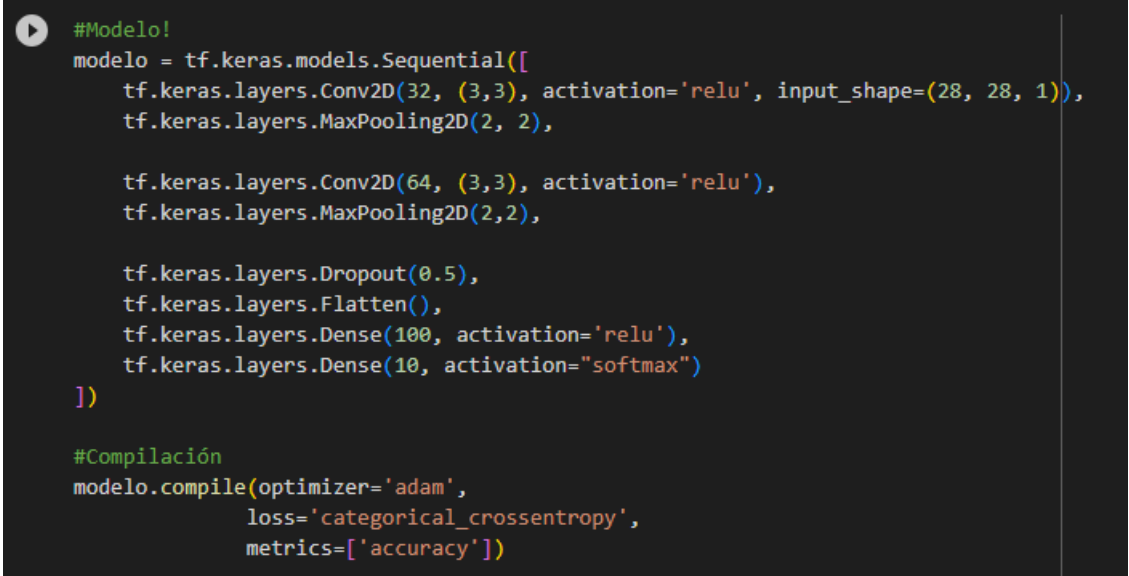
Figura 6. Creación de los conjuntos de datos

Desarrollo de la red neuronal

Una vez preprocesados y preparados los datos, se ha empezado con el desarrollo de la red neuronal convolucional que será la encargada de realizar la clasificación.

La red neuronal convolucional que se ha utilizado de base se encuentra en el siguiente enlace de Google Colab: <https://colab.research.google.com/drive/1NUWKrlDDaLS-K539TGEsRk9ch28YDGCm?usp=sharing>.

La configuración inicial de la red neuronal que se muestra en el Colab se puede observar en la Figura 7.



```
#Modelo!
modelo = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(28, 28, 1)),
    tf.keras.layers.MaxPooling2D(2, 2),

    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),

    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation='relu'),
    tf.keras.layers.Dense(10, activation="softmax")
])

#Compilación
modelo.compile(optimizer='adam',
               loss='categorical_crossentropy',
               metrics=['accuracy'])
```

Figura 7. Red neuronal de partida [9]

Para la realización de los entrenamientos se ha creado un cuaderno Jupyter con el código a ejecutar, también se cuenta con un fichero Python donde se encuentran todas las funciones de carga y preparación de las imágenes que se utilizarán para el entrenamiento y las funciones de callback utilizadas para ver la mejora del entrenamiento cada N iteraciones.

El cuaderno Jupyter empieza con la carga del dataset. En primer lugar, se accederá a los distintos directorios (los nombres de los directorios se definirán en una lista) donde se encuentran las imágenes de cada tipo de trufa, cada imagen será reescalada a un tamaño de 90x70, las imágenes se guardarán en tuplas compuestas por la imagen y el nombre de la categoría a la que pertenecen junto al número de la imagen (Un ejemplo del nombre sería 'TuberAestivum04'). El código correspondiente se puede ver en la Figura 8.

```

def cargaImagenes(directorio, categorias):
    data = []
    for categoria in categorias:
        try:
            path = os.path.join(directorio,categoria)
            listdir = os.listdir(path)
        except Exception as e:
            print('Error al cargar directorios')
            listdir = []
            path='Error'

        for i in tqdm.tqdm(range(len(listdir)), desc=categoria):
            imagen_nombre = listdir[i]
            try:
                imagen_ruta = os.path.join(path,imagen_nombre)
                imagen = cv2.imread(imagen_ruta)
                imagen = cv2.resize(imagen,(90, 70))
                imagen_nombre = imagen_nombre.replace('.jpg','')
                data.append([imagen, categoria+imagen_nombre])

            except Exception as e:
                print('Error al cargar las imágenes')

    return data

```

Figura 8. Carga de imágenes

Una vez cargadas las imágenes, se prepararán para poder ser utilizadas. Para ello, se tendrán que separar las imágenes de las etiquetas correspondientes a cada imagen.

Para ello, se obtienen las categorías a partir del nombre de cada imagen (obtenido de la tupla creada en la Figura 8). Una vez obtenida la categoría de la imagen, se crearán dos listas: una que almacenará las imágenes y otra que almacenará las etiquetas asignadas a cada imagen. En este paso también se harán las cinco rotaciones aleatorias y el modo espejo mencionado anteriormente en la Figura 5.

Una vez cargadas las imágenes y las etiquetas se guardarán como una nueva tupla en una lista auxiliar y se mezclarán aleatoriamente para evitar la generación de patrones. Por último, se volverán a separar las imágenes y las etiquetas en dos vectores independientes.

```

#Prepara los datos para ser utilizados, asigna la categoría a cada imagen.
#Mezcla aleatoriamente el conjunto de imágenes para no generar patrones.
def preparaDatos(data):
    imagenes = []
    labels = []
    im_aux = []

    for i in range(len(data)):
        imagen = data[i]
        cat = imagen[1]
        res = re.findall('(\d+|[A-Za-z]+)', cat)
        img = imagen[0]
        labels.append(res[0])
        imagenes.append(img)

        for j in [1,1,1,1,1]:
            img_rotada = rotaImagen(img)
            imagenes.append(img_rotada)
            labels.append(res[0])

        img_espejo = np.flip(img,axis=1)
        labels.append(res[0])
        imagenes.append(img_espejo)

    for i in range(len(imagenes)):
        im_aux.append((imagenes[i],labels[i]))

    rn.shuffle(im_aux)

    imagenes.clear()
    labels.clear()

    for i in range(len(im_aux)):
        dat = im_aux[i]
        ima = dat[0]
        lab = dat[1]
        imagenes.append(ima)
        labels.append(lab)

    return imagenes, labels

```

Figura 9. Preparación de las imágenes

Ahora que todas las imágenes tienen asociada una categoría, se ha realizado la conversión de cada categoría a un valor numérico para facilitar así el entrenamiento de la red neuronal. De esta manera se pasará de tener una lista de categorías como esta [TuberAestivum, TuberBorchii, TuberAestivum, TuberRufum, ...] a una lista como ésta [0, 1, 0, 16, ...].

Una vez transformadas las categorías, se llama a la función mencionada en la Figura 6. En esta función se crearán los conjuntos de entrenamiento y test y se normalizarán los datos. Una vez creados los conjuntos, se llamará a la función 'to_categorical' de la librería de Keras, to_categorical convierte las etiquetas enteras en vectores one-hot. Un vector one-hot es un vector binario en el que un solo elemento es "1" y todos los demás son "0". Cada posición en el vector corresponde a una clase, y el "1" se coloca en la posición correspondiente a la clase a la que pertenece la etiqueta. Este paso se realizará tanto en las etiquetas del conjunto de entrenamiento como en las del conjunto de test como se observa en la Figura 10.

```
#Cambio las etiquetas de codificación categórica a one-hot  
train_Y_one_hot = to_categorical(train_Y)  
test_Y_one_hot = to_categorical(test_Y)
```

Figura 10. Cambio de etiquetas a one-hot

En la Figura 11 se observa un ejemplo del cambio de etiquetas categóricas a one-hot y la división de las imágenes en conjunto de entrenamiento y conjunto de test.

```
Original label: 11  
After conversion to one-hot: [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]  
(1042, 70, 90, 3) (35, 70, 90, 3) (1042, 18) (35, 18)
```

Figura 11. Ejemplo de one-hot

A la hora de realizar el entrenamiento, se ha implementado una función de validación (o Callback en inglés) para ver los distintos resultados a lo largo del entrenamiento. Esta función elegirá una imagen al azar y se llamará a la función *predict* de la librería de Keras para obtener la predicción del tipo de trufa que el modelo entrenado identifica en la imagen seleccionada. Cada N iteraciones(definidas en la propia función) se ejecutará esta función de validación. Los resultados se almacenarán en un fichero CSV con el número de la iteración, etiqueta real y etiqueta predicha por el modelo. Se pueden ver las funciones creadas en la Figura 12.

```
def custom_validation_function(model, x_val, y_val):
    # Selecciona una imagen aleatoriamente
    idx = np.random.randint(0, len(x_val))
    image = np.expand_dims(x_val[idx], axis=0)
    true_label = np.argmax(y_val[idx])

    # Predice la clasificación
    prediction = model.predict(image)
    predicted_label = np.argmax(prediction)

    return true_label, predicted_label

class CustomValidationCallback(Callback):
    def __init__(self, x_val, y_val, csv_path, validation_frequency=1000):
        super(CustomValidationCallback, self).__init__()
        self.x_val = x_val
        self.y_val = y_val
        self.csv_path = csv_path
        self.validation_frequency = validation_frequency
        self.iteration = 0

    def on_batch_end(self, batch, logs=None):
        self.iteration += 1
        if self.iteration % self.validation_frequency == 0:
            true_label, predicted_label = custom_validation_function(self.model, self.x_val, self.y_val)

            # Guarda los resultados en un CSV
            with open(self.csv_path, 'a') as f:
                f.write(f"{self.iteration},{true_label},{predicted_label}\n")
```

Figura 12. Funciones de validación

Ahora que están definidas todas las funciones que se van a utilizar, se va a pasar a crear la red neuronal con sus distintas capas. Después de distintas pruebas que se detallarán en el siguiente apartado, se ha optado por la siguiente configuración de la red neuronal (Figura 13).

```

truqu_model = Sequential()

truqu_model.add(Conv2D(128, kernel_size=(7,7), activation='linear', padding='same', input_shape=(70,90,3)))
truqu_model.add(LeakyReLU(alpha=0.1))
truqu_model.add(MaxPooling2D(pool_size=(2, 2), padding='same'))
truqu_model.add(Dropout(0.5))

truqu_model.add(Conv2D(64, kernel_size=(7,7), activation='linear', padding='same', input_shape=(70,90,3)))
truqu_model.add(LeakyReLU(alpha=0.3))
truqu_model.add(MaxPooling2D(pool_size=(2, 2), padding='same'))
truqu_model.add(Dropout(0.5))

truqu_model.add(Conv2D(64, kernel_size=(3,3), activation='linear', padding='same', input_shape=(70,90,3)))
truqu_model.add(LeakyReLU(alpha=0.1))
truqu_model.add(MaxPooling2D(pool_size=(2, 2), padding='same'))
truqu_model.add(Dropout(0.5))

truqu_model.add(Flatten())
truqu_model.add(Dense(64, activation='linear'))
truqu_model.add(LeakyReLU(alpha=0.1))
truqu_model.add(Dropout(0.5))
truqu_model.add(Dense(nClasses, activation='softmax'))

```

Figura 13. Configuración de la red neuronal

La red neuronal cuenta con las siguientes capas:

- Capa convolucional (Conv2D) que utiliza 128 filtros con un tamaño de kernel de 7x7, activación lineal, relleno 'same' que mantendrá el tamaño de la salida igual al de la entrada, con una entrada (70,90,3) que son imágenes de 90x70 con 3 canales de color(RGB). A esta capa se le aplica una función de activación Leaky ReLU con alpha=0.1 y una capa MaxPooling2D con un tamaño de 2x2 para reducir la dimensionalidad espacial de la entrada y por último una capa de Dropout con una tasa del 50% para evitar el sobreajuste.
- Capa convolucional (Conv2D) similar a la primera, pero con 64 filtros y una activación Leaky ReLU con un parámetro alpha de 0.3.
- Otra capa convolucional (Conv2D) con 64 filtros de tamaño 3x3 y activación Leaky ReLU con un parámetro alpha de 0.1.
- Capa Flatten para transformar la salida de la última capa convolucional en un vector unidimensional.
- Capa totalmente conectada (Dense) con 64 neuronas y activación lineal y Leaky ReLU con parámetro alpha de 0.1 y una capa de Dropout.
- Por último, capa de salida (Dense) con activación 'softmax', con número de neuronas igual al número de tipos de trufas elegidos (18 en total).

Creación del dataset

En primer lugar, se ha realizado un estudio acerca de los distintos tipos de trufas que existen en Europa. Para este trabajo se utilizarán únicamente tipos de trufas que puedan ser recolectados en países europeos.

Para cada tipo de trufa se ha recopilado la siguiente información:

- Nombre común.
- Nombre técnico.
- Fecha o temporada de recolección de la trufa.
- Color, forma y tamaño.
- Sabor y olor.
- Una imagen de la trufa(tanto por dentro como por fuera)

La información de cada tipo de trufa se ha ido encontrando por distintas páginas webs especializadas en el ámbito de la micología [4] [10] [23] [7].

Una vez recolectada toda la información, se ha creado un fichero ([Excel](#)) en el que se recoge toda la información obtenida.

En segundo lugar, se han buscado imágenes de los tipos de trufas encontrados anteriormente. Se han buscado imágenes tanto del exterior como del interior de las trufas ya que existen especies de trufas que en el exterior son iguales, pero se diferencian en el interior y viceversa.

Una vez encontradas las imágenes, se han almacenado de manera local y se han buscado distintas tecnologías para implementar un dataset. La herramienta elegida para crear el dataset ha sido Google BigQuery. A continuación, se detalla el proceso de creación del dataset:

1. Se ha accedido a Google Cloud [8] y se ha creado un nuevo proyecto. Una vez creado el proyecto se han habilitado las APIs de Google BigQuery y BigQuery Connection, esta segunda API permitirá realizar la conexión entre BigQuery y el almacén de Cloud Storage que se utilizará para almacenar las imágenes.

2. Se ha creado un conjunto de datos o esquema:
 - `CREATE SCHEMA `<<id_proyecto>>.data_set`;`
3. Una vez creado el esquema, se agregará la conexión a los datos, para ello se ha pulsado en “AGREGAR” y luego se ha seleccionado la opción de “Conexión a fuentes de datos externas” (Figura 14).



Una vez seleccionada esa opción, se ha configurado la conexión a la fuente externa como se muestra en la Figura 15.

Fuente de datos externa

Tipo de conexión
BigLake y funciones remotas (Recurso de Cloud)

ID de conexión *
data-conn

Tipo de ubicación ?

☐ Región
Especifica una región para colocar tus conjuntos de datos con otros servicios de Google Cloud.

☒ Multirregional
Permite que BigQuery seleccione una región de un grupo para alcanzar límites de cuota más altos.

Multirregional *
EU (varias regiones en la Unión Europea)

Nombre descriptivo

Descripción

Figura 15. Configuración de fuente externa

4. Con la conexión ya creada, se pasará a la creación del Cloud Storage. En este caso se utilizará un Bucket, se accederá a Cloud Storage y seleccionando “Crear Bucket”, se le da un nombre, y se selecciona la región en la que se van a almacenar los datos (Figura 16).

← Crear un bucket

☒ **Asigna un nombre a tu bucket**
Nombre: truqu-images

☒ **Elige dónde almacenar tus datos**
Ubicación: eu (varias regiones en la Unión Europea)
Tipo de ubicación: Multi-region

☒ **Elige una clase de almacenamiento para tus datos**
Clase de almacenamiento predeterminada: Standard

☒ **Elige cómo controlar el acceso a los objetos**
Prevención del acceso público: Desactivada
Control de acceso: Uniforme

Figura 16. Creación del bucket

5. Una vez creado el Bucket, se almacenarán en él todas las imágenes de trufas separadas por tipos. Con los datos ya añadidos al Bucket , habrá que permitir la conexión entre el esquema creado anteriormente y mediante el cual se realizarán las consultas y el bucket donde se almacenan las imágenes, para ello se introducirán los siguientes comandos:

```
- export sa=data-conn
- gsutil iam ch serviceAccount:$sa:objectViewer "gs://truqu-images "
```

Figura 17. Código para dar permiso de conexión

Se creará una tabla en BigQuery para poder acceder a los datos. Se ejecutará la siguiente consulta:

```
CREATE OR REPLACE EXTERNAL TABLE `truqu.data_set.truqu-types`
WITH CONNECTION `eu.data-conn`
OPTIONS(
  object_metadata="SIMPLE", uris=["gs://truqu-images/*.jpg"]);
```

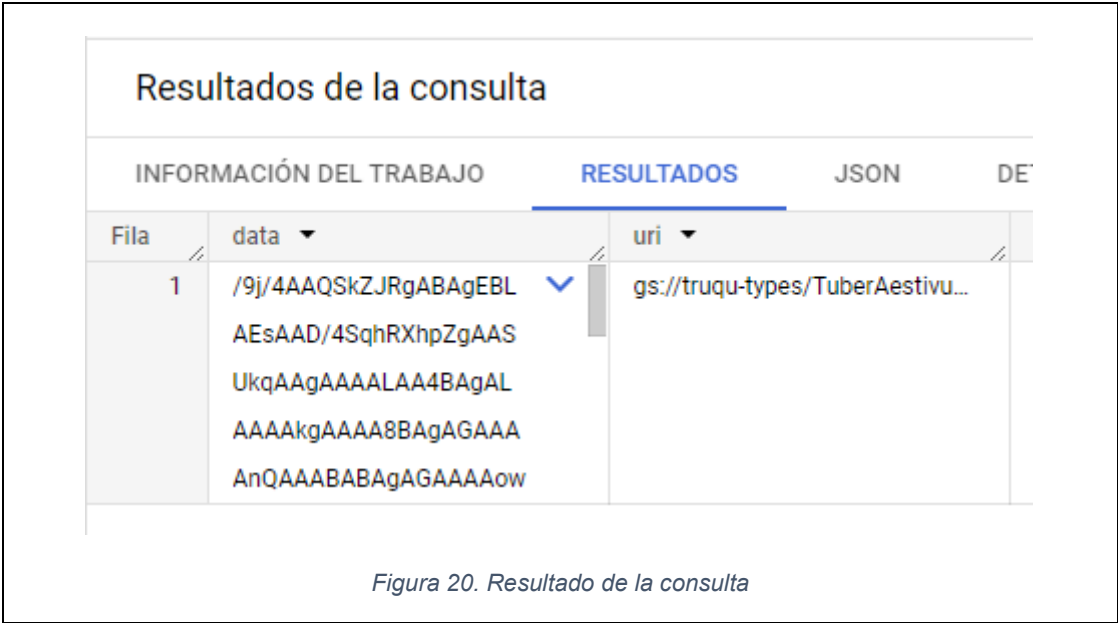
Figura 18. Consulta SQL para crear conexión entre Bucket y BigQuery

Una vez creada la tabla mediante la que se podrá acceder a los datos del Cloud Storage Bucket en el que se han almacenado las imágenes, se podrán ejecutar consultas SQL (en inglés, Structured Query Language) para obtener las imágenes, como se observa en la Figura 19.

```
1 SELECT data , uri
2 FROM `data_set.truqu-types`
3 WHERE REGEXP_CONTAINS(uri, 'gs://truqu-images/TuberAestivum')
4 Limit 1;
```

Figura 19. Consulta SQL para obtener una imagen

Una vez ejecutada la consulta anterior, se obtendrá la referencia a una imagen del tipo de trufa Tuber Aestivum, como se observa en la Figura 20.



Los datos obtenidos con la consulta pueden ser exportados de manera sencilla en diferentes formatos (CSV y JSON), y posteriormente, la imagen seleccionada puede verse mediante Python.

Pruebas realizadas

En este apartado se abordará en detalle el análisis de las pruebas realizadas para la obtención de resultados. Se explorarán diversos aspectos clave, comenzando con la descripción de las pruebas realizadas utilizando Cloud Vision AI. A continuación, se examinará la influencia de los hiperparámetros seleccionados en el desempeño general del sistema. Posteriormente, se presentarán los resultados de pruebas específicas llevadas a cabo en entornos con CPU y GPU, y se realizará una comparativa detallada entre el desempeño de CPU y GPU, proporcionando una visión clara de las ventajas y limitaciones de cada plataforma. Finalmente, se expondrá y evaluará el modelo definitivo.

Cloud Vision AI

Una vez creado el dataset, se investigaron otros softwares que permiten reconocer elementos en imágenes. Por ejemplo, Google cuenta con Cloud Vision AI, una API que permite realizar distintos procesos de detección y reconocimiento en imágenes. Cloud Vision cuenta con una demo sencilla de usar en su página web [25].

Esta demo, a partir de una imagen dada, permite obtener características como ver que objetos se encuentran en esa imagen, los colores predominantes, reconocimiento de texto en la imagen, entre otras funcionalidades.

Para probar las funcionalidades de esta API, se le ha pasado una imagen de una trufa de tipo 'Tuber Excavatum' y una imagen de unas patatas para comparar la respuesta de la API al recibir una imagen de un objeto común como es una patata y al recibir una imagen de un objeto no tan común como es la trufa. Se han obtenido los resultados que se observan en la Figura 21 y Figura 22.

Mediante la funcionalidad de identificación de objetos (Figura 21) se puede observar que al recibir la imagen de la trufa los resultados devueltos son "Food", un resultado genérico ya que la API no es capaz de identificar la trufa, mientras que al recibir una imagen de una patata la API sí que es capaz de reconocer que el objeto de la imagen es una patata.

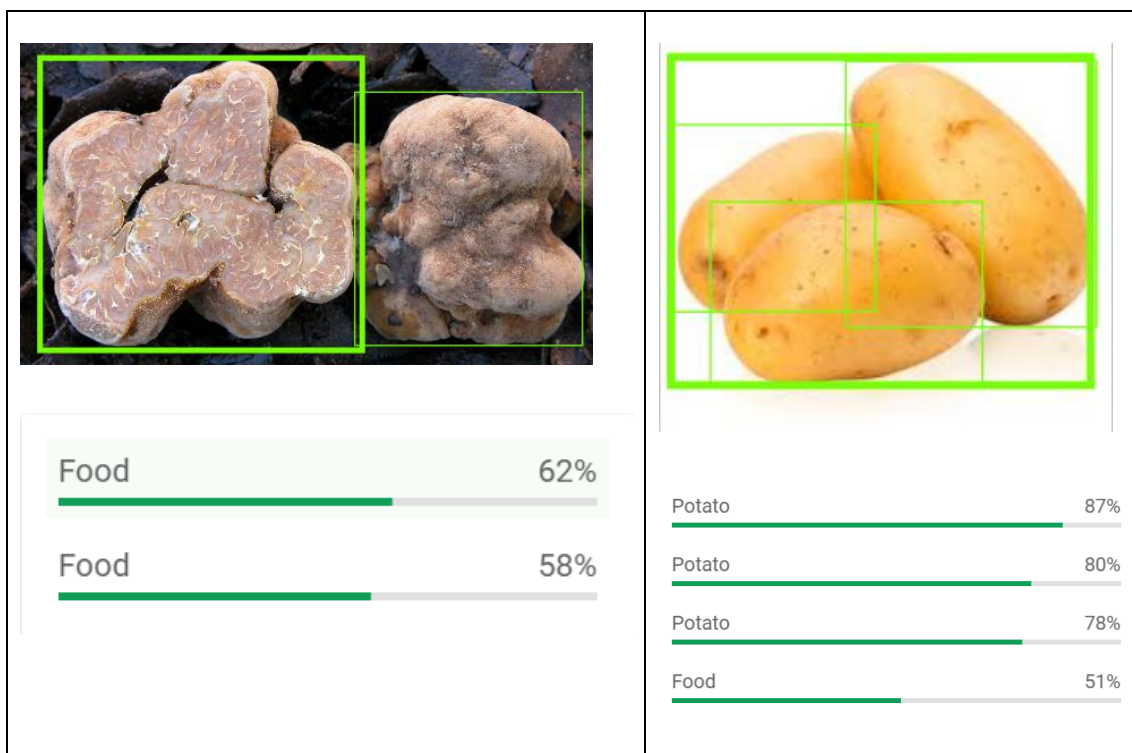


Figura 21. Comparativa usando Vision AI

En la segunda prueba, se ha probado a utilizar la funcionalidad de detección de etiquetas, esta funcionalidad analiza la imagen y devuelve etiquetas relacionadas con el contenido de la imagen. En este caso, al introducir la imagen de la trufa, se observa que los resultados obtenidos son 'Naturaleza', 'Roca' y 'Madera', lo que indica que no está identificando correctamente el contenido de la imagen. En cambio, al introducir la imagen de la patata, los resultados obtenidos son 'Comida', 'Patata', 'Ingrediente' y 'Comida natural', se observa que en este caso sí que ha identificado que la imagen era una patata y ha devuelto etiquetas acordes a la imagen.

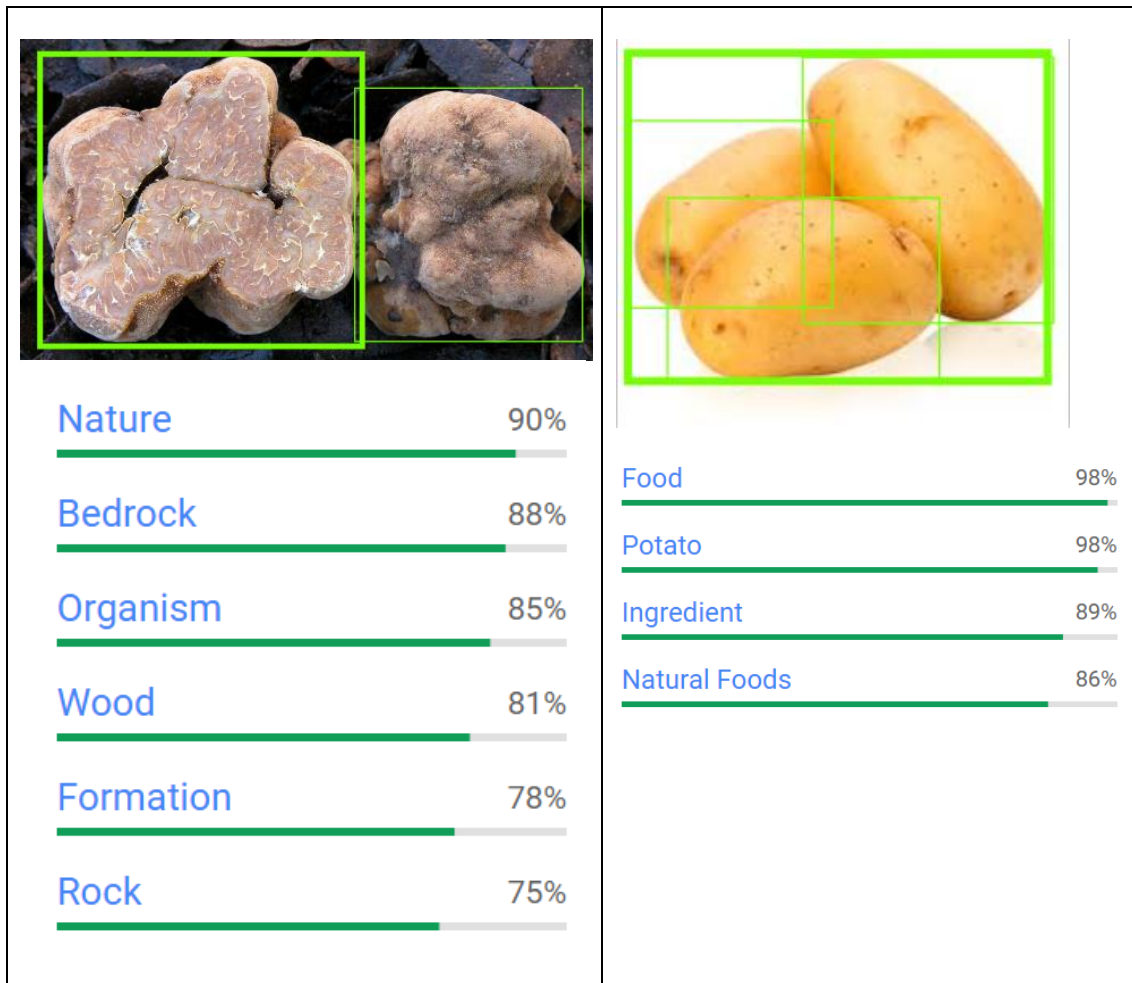


Figura 22. Comparativa usando Vision AI

Realizadas estas pruebas, se ha descartado la utilización de esta API ya que no satisface las necesidades de este proyecto, puesto que no es ni siquiera capaz de reconocer una trufa. Una vez descartada la opción de Google Vision AI, se ha optado por la creación de la red neuronal explicada anteriormente (Figura 13). Con la red neuronal creada, se han realizado pruebas utilizando CPU y GPU para ver en qué entorno el entrenamiento es más efectivo.

Hiperparámetros

Los hiperparámetros son las características de una red neuronal que se establecen manualmente antes del procesamiento de los datos. Estos determinan cómo la red gestionará los datos de entrada y afectará al comportamiento de la misma.

Para la realización de las pruebas se han ido modificando algunos **hiperparámetros** de la red neuronal como el tamaño de batch (*batch size* en inglés), el cual define el número de muestras que se introducen en la red para que sean procesadas simultáneamente. Si este valor es pequeño, significa que la red tiene en memoria pocas imágenes, y por lo tanto el entrenamiento es más rápido. Sin embargo, es posible que no se trabajen adecuadamente las características y detalles de las imágenes, y, por lo tanto, el resultado sea peor. Por otro lado, si este valor es grande, el entrenamiento será más lento, pero la red será capaz de extraer un conjunto de características más amplio de cada imagen.

Los valores que se suelen utilizar en este hiperparámetro son potencias de 2, por lo que las pruebas se van a realizar con un tamaño de batch de 1, 8, 16 y 32, para medir el entrenamiento de cada prueba se va a utilizar la función de validación mencionada anteriormente (Figura 12). Se ha configurado la función de validación de manera que cada 1000 iteraciones se ponga en funcionamiento y haga una comprobación, el estudio se ha realizado con 500 validaciones por prueba.

Pruebas utilizando GPU

En la Figura 23 se observa una gráfica que compara las 4 pruebas realizadas, en este caso utilizando GPU. En el eje vertical de la gráfica se presenta el número de aciertos acumulado mientras que en el eje horizontal se presenta el número de validaciones, como se ha mencionado anteriormente, se han realizado 500 validaciones por entrenamiento.

Como se observa en la gráfica, la prueba que más aciertos ha obtenido es la realizada con un tamaño de batch = 32, con 300 aciertos en 500 validaciones, la prueba con tamaño de batch = 16 es la segunda con alrededor de 240 aciertos en 500 validaciones y, por último, las pruebas con tamaño de batch = 1 y = 8 cuentan con resultados bastante parejos con un total de 170-180 aciertos en 500 validaciones.

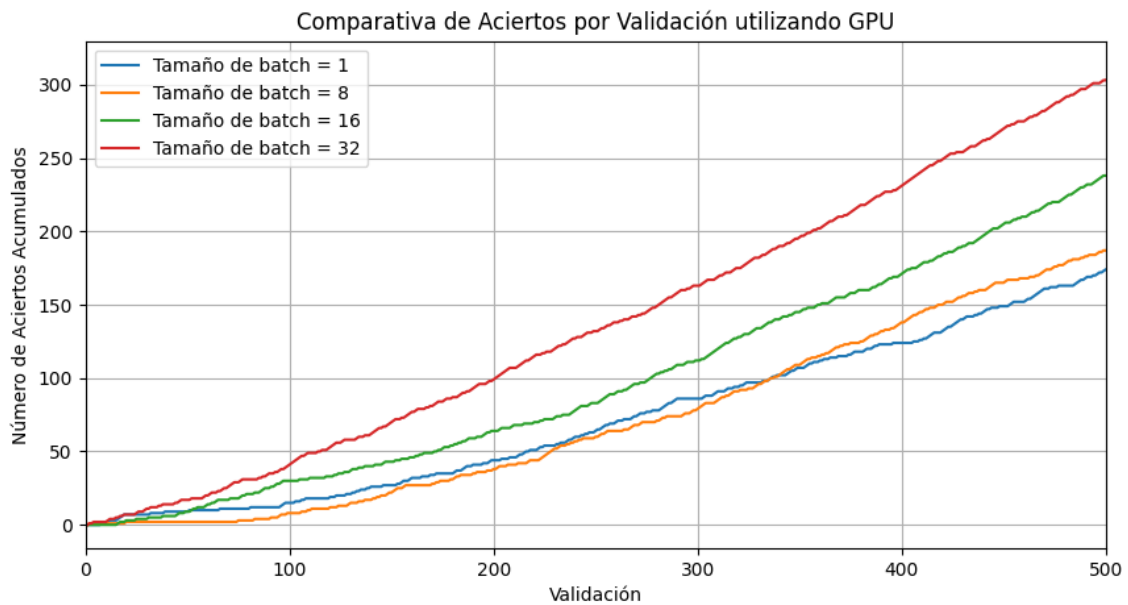


Figura 23. Pruebas con GPU

Pruebas utilizando CPU

En la Figura 24 se observa una gráfica que compara las 4 pruebas realizadas, en este caso utilizando GPU. En el eje vertical de la gráfica se presenta el número de aciertos acumulado mientras que en el eje horizontal se presenta el número de validaciones, como se ha mencionado anteriormente, se han realizado 500 validaciones por entrenamiento.

Como se observa en la gráfica, la prueba que más aciertos ha obtenido es la realizada con un tamaño de batch = 32, con alrededor de 280 aciertos en 500 validaciones, la prueba con tamaño de batch = 16 es la segunda con alrededor de 210 aciertos en 500 validaciones y, por último, las pruebas con tamaño de batch = 1 y = 8 cuentan con resultados bastante parejos con un total de 140-160 aciertos en 500 validaciones.

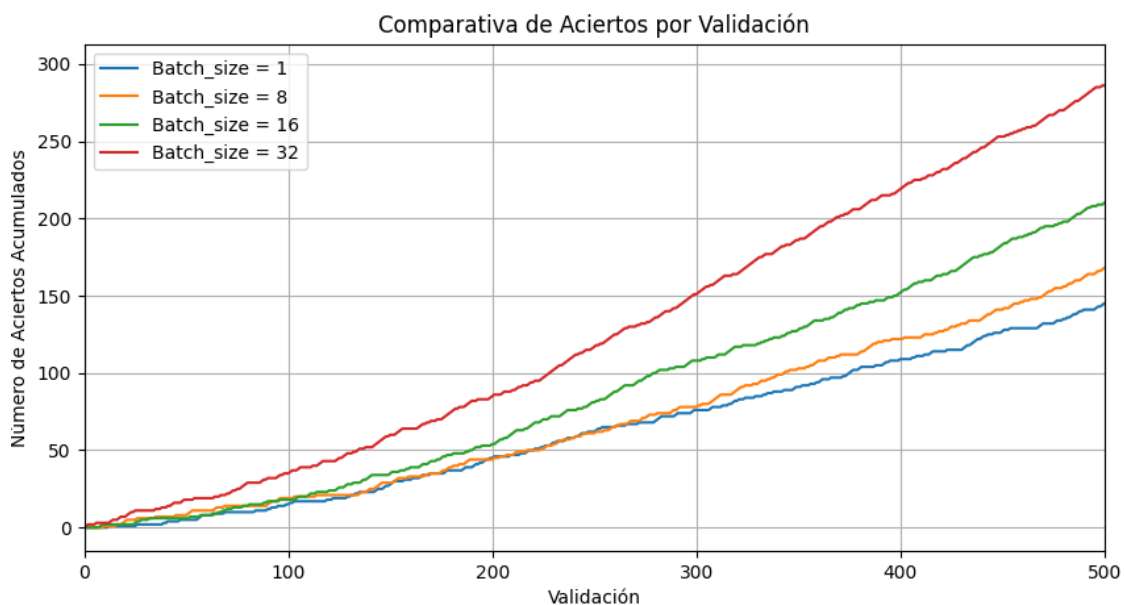


Figura 24. Pruebas con CPU

Comparativa entre CPU y GPU

Una vez obtenidas las gráficas con los resultados de los entrenamientos utilizando tanto CPU como GPU, se observa que en ambas pruebas el mejor resultado se ha obtenido al utilizar un tamaño de batch = 32, esto se debe a que al utilizar lotes más grandes se estabiliza la actualización de los pesos eliminando así el 'ruido' en el entrenamiento. Otra razón por la que utilizar un tamaño de batch = 32 es que a la hora de aprovechar los recursos de la máquina al máximo, en términos de eficiencia computacional, el uso de tamaños de lote más grandes permite procesar más ejemplos en paralelo, lo que puede ser beneficioso para la velocidad de entrenamiento.

Comparando ahora los 2 mejores entrenamientos entre CPU y GPU, como se observa en la Figura 25, ambos resultados son parejos siendo el entrenamiento con GPU un poco más efectivo ya que obtiene un mayor número de aciertos.

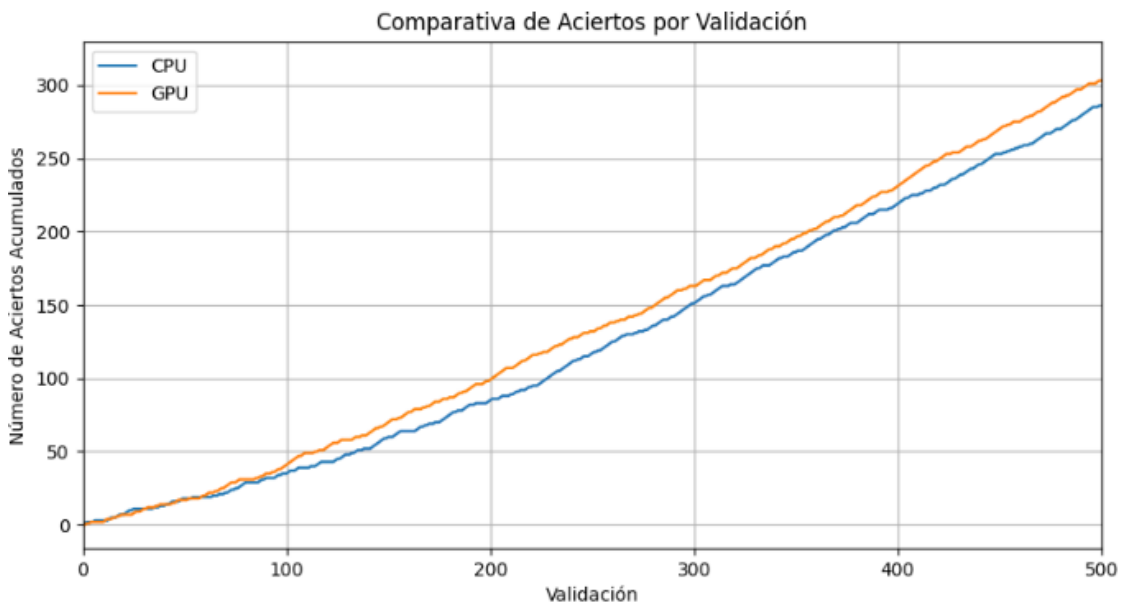


Figura 25. Comparativa de entornos

En cuanto a los tiempos de ejecución, se observa que procesar el entrenamiento utilizando GPU es mucho más rápido que procesar el entrenamiento con CPU, las GPUs están diseñadas para tareas altamente paralelas, como las requeridas en el entrenamiento de redes neuronales, y, por lo tanto, ofrecen un rendimiento significativamente superior en comparación con las CPUs para estas tareas específicas. Además, las bibliotecas CUDA (Compute Unified Device Architecture) y cuDNN (CUDA Deep Neural Network library) utilizadas por tensorflow-gpu proporcionan interfaces y optimizaciones específicas para GPU que aceleran las operaciones comunes en aprendizaje profundo.

	Batch = 1	Batch = 8	Batch = 16	Batch = 32
CPU	226 minutos	500 minutos	2200 minutos	2730 minutos
GPU	40 minutos	68 minutos	114 minutos	187 minutos

Tabla 3. Comparativa de tiempos de las pruebas

Una vez analizadas las opciones de CPU y GPU y los tiempos de entrenamiento de cada entorno, se ha observado que el entrenamiento en el entorno de GPU es más favorable en cuanto a porcentaje de acierto y muy favorable en cuanto a tiempos de ejecución.

Modelo final

Una vez realizadas las comparativas entre los entrenamientos realizados con GPU frente a los entrenamientos realizados con CPU, se ha optado por realizar un entrenamiento más extenso para conseguir una red neuronal que sea capaz de predecir de manera consistente los distintos tipos de trufas.

Se ha decidido realizar las pruebas utilizando la GPU ya que ofrecen mejores resultados y además el tiempo de entrenamiento es considerablemente más rápido, se han realizado pruebas con 30000 épocas (en inglés, epochs) con un tamaño de batch = 32, esto quiere decir que se ejecutarán 30000 épocas de entrenamiento utilizando lotes de 32 ejemplos de entrenamiento en cada época de entrenamiento.

En primer lugar, las primeras pruebas se han realizado con una tasa de aprendizaje (en inglés, learning rate) estática de $1e-3$, después de algunas pruebas, en la gráfica de la Figura 26 se observa el resultado del mejor modelo obtenido utilizando esta configuración. Como se puede observar, en un total de alrededor de 1000 validaciones realizadas, se han obtenido un número de aciertos de alrededor de 425 aciertos en total, lo que indica que este modelo no es muy consistente ya que falla en un 60% de los reconocimientos que hace. A partir de la validación número 500, el entrenamiento ha empezado a empeorar y a no crecer de manera correcta.

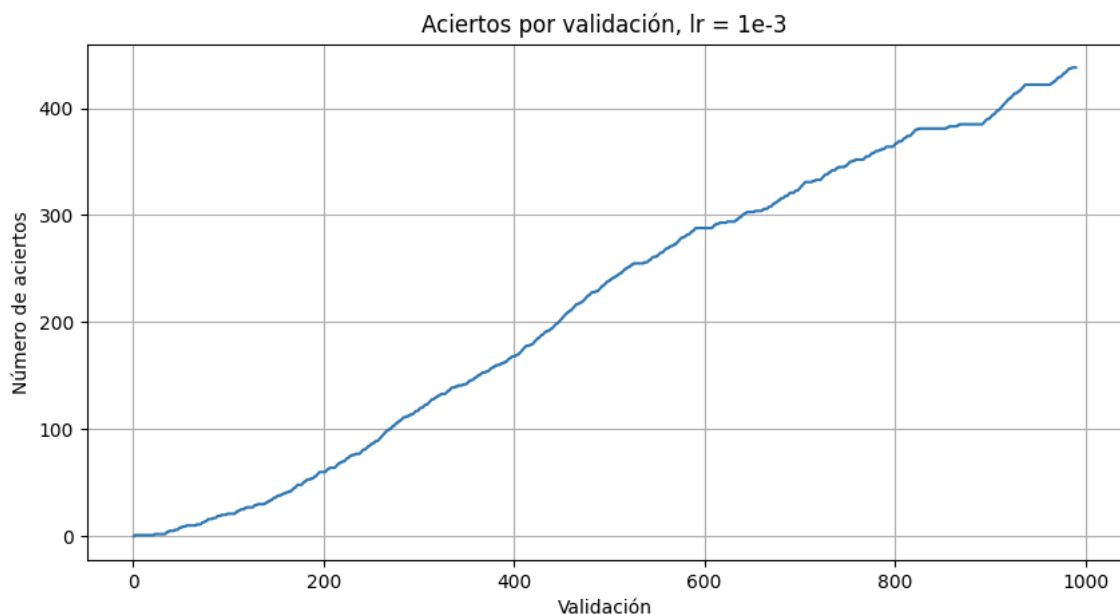


Figura 26. Gráfica del entrenamiento con lr fijo

Posteriormente, se han realizado una serie de pruebas sustituyendo el learning rate fijo por un learning rate variable cada N iteraciones, para este entrenamiento se ha utilizado una configuración programada de manera que cada N iteraciones que realice el entrenamiento, se pueda modificar el learning rate y así conseguir un entrenamiento más consistente. Para conseguir este learning rate variable se han utilizado las instrucciones que se muestran en la Figura 27.

```
#Ajusta el learning rate de manera variable
step = tf.Variable(0, trainable=False)
schedule = tf.optimizers.schedules.PiecewiseConstantDecay(
    [400000, 650000], [1e-3, 1e-4, 1e-5])
lr = 1e-3 * schedule(step)
epochs = 30000
batch_size = 32
```

Figura 27. Learning rate variable

El conjunto de datos total de entrenamiento es de 1042 imágenes, por lo que dividiendo 1042/32 (número de imágenes / tamaño de los lotes) se obtendrán un total de 33 imágenes por lote. Si ahora se multiplica estas 33 imágenes por bloque por el número de épocas, se obtendrán un total de 990000 iteraciones totales de entrenamiento.

Este número es importante para segmentar el learning rate variable que se ha comentado anteriormente. Como se observa en la Figura 27, se ha creado una variable 'schedule' que determinará donde se realiza el cambio entre un learning rate y el siguiente. En este caso, el primer cambio se realiza en la iteración 400000, lo que significa que desde la iteración 0 hasta la iteración 400000 se utilizará un learning rate de 1e-3, desde la iteración número 400000 hasta la iteración número 650000 se utilizará un learning rate de 1e-4 y a partir de la iteración 650000 hasta el final de la ejecución se utilizará un learning rate de 1e-5. En redes neuronales, el learning rate es un **hiperparámetro clave** ya que determina los ajustes que se realizan en los pesos de la red durante el entrenamiento.

La gráfica de la Figura 28 representa los resultados del mejor modelo obtenido con learning rate variable. Como se puede observar, este entrenamiento ha conseguido un número mayor de aciertos con el mismo número de validaciones, lo que significa que será más consistente a la hora de realizar el reconocimiento de las trufas, por otra parte, el crecimiento de los aciertos a medida que se van realizando validaciones es más regular que el visto en la gráfica de la Figura 26, lo que indica que el entrenamiento ha ido mejorando exponencialmente de manera correcta.

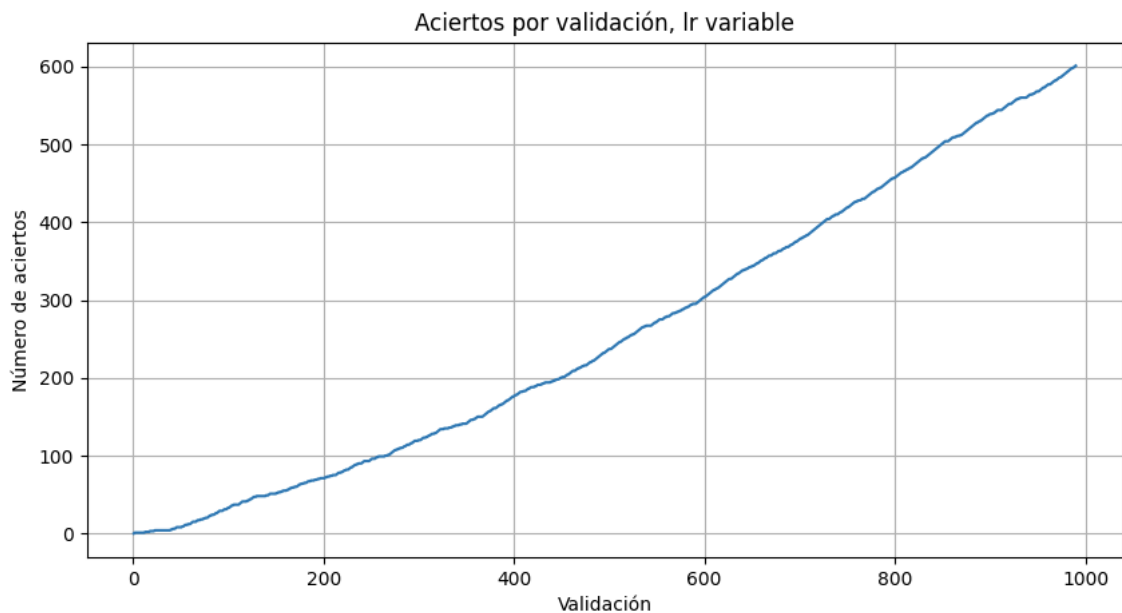


Figura 28. Gráfica del entrenamiento con lr variable

A continuación, se muestra una gráfica que compara los resultados de los 2 modelos mencionados anteriormente (Figura 29).

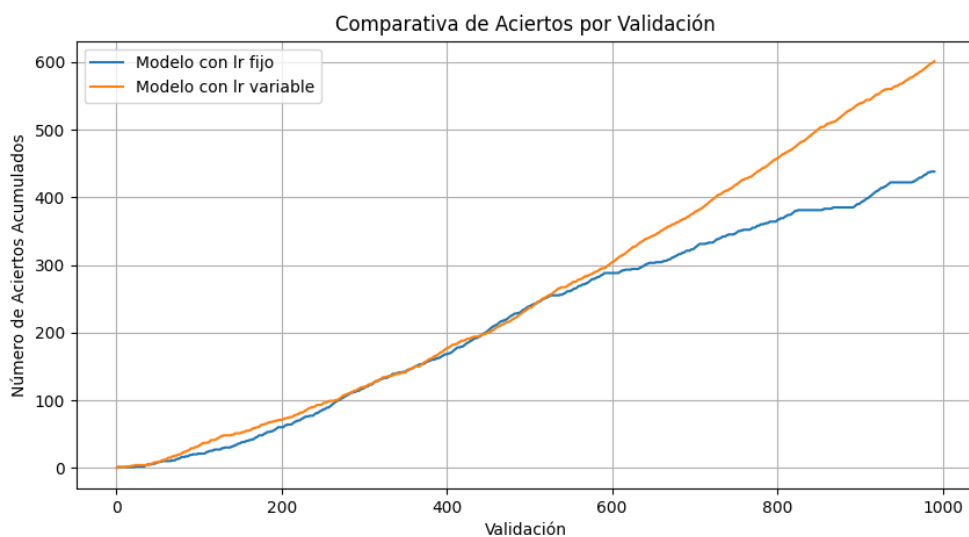


Figura 29. Comparativa de modelos

Con los modelos ya entrenados, se ha utilizado el código mostrado en la Figura 30 para comprobar la precisión de cada uno de ellos. Para ello se utiliza la función *predict* proveniente de la librería *Keras*. Mediante esta función se obtiene un porcentaje predicho de cada tipo de trufa, de manera que la categoría que más porcentaje tenga es la predicha por el modelo. Para que los resultados se puedan interpretar de una mejor manera, se ha obtenido la categoría predicha y se ha comparado con la categoría real de la imagen, si éstas coinciden se sumará el acierto.

```
modelo = tf.keras.models.load_model('/content/modelo.h5py')

test_eval = modelo.predict(test_X)
acertadas = 0
i = 0
for pred in test_eval:
    mayor = max(pred)

    posicion = np.where(pred==mayor)[0][0]

    print('Clasificacion real: ', test_Y[i], 'Clasificacion predicha: ', posicion, ' Porcentaje -->', mayor)
    if test_Y[i] == posicion:
        acertadas += 1
    i += 1

print('Porcentaje acierto: ', acertadas/len(test_Y)*100)
```

Figura 30. Código para probar modelo entrenado

El modelo con learning rate fijo ha dado los resultados que se observan en la Figura 31. El porcentaje de acierto es del 66.6%, lo que quiere decir que este modelo no cuenta con una gran fiabilidad y no es consistente a la hora de realizar el reconocimiento.

```

Clasificacion real: 5 Clasificacion predicha: 5 Porcentaje --> 0.65862495
Clasificacion real: 1 Clasificacion predicha: 12 Porcentaje --> 0.28464717
Clasificacion real: 8 Clasificacion predicha: 9 Porcentaje --> 0.44689667
Clasificacion real: 17 Clasificacion predicha: 7 Porcentaje --> 0.20499203
Clasificacion real: 14 Clasificacion predicha: 14 Porcentaje --> 0.19320676
Clasificacion real: 3 Clasificacion predicha: 3 Porcentaje --> 0.28272176
Clasificacion real: 5 Clasificacion predicha: 9 Porcentaje --> 0.19895418
Clasificacion real: 7 Clasificacion predicha: 7 Porcentaje --> 0.9457567
Clasificacion real: 12 Clasificacion predicha: 12 Porcentaje --> 0.16989452
Clasificacion real: 1 Clasificacion predicha: 1 Porcentaje --> 0.24536401
Clasificacion real: 0 Clasificacion predicha: 0 Porcentaje --> 0.26079342
Clasificacion real: 0 Clasificacion predicha: 0 Porcentaje --> 0.5207868
Clasificacion real: 0 Clasificacion predicha: 0 Porcentaje --> 0.4138125
Clasificacion real: 7 Clasificacion predicha: 7 Porcentaje --> 0.23261635
Clasificacion real: 4 Clasificacion predicha: 4 Porcentaje --> 0.50490093
Clasificacion real: 3 Clasificacion predicha: 3 Porcentaje --> 0.45363027
Clasificacion real: 1 Clasificacion predicha: 1 Porcentaje --> 0.16904445
Clasificacion real: 3 Clasificacion predicha: 3 Porcentaje --> 0.13086878
Clasificacion real: 8 Clasificacion predicha: 10 Porcentaje --> 0.27712652
Clasificacion real: 15 Clasificacion predicha: 15 Porcentaje --> 0.57031107
Clasificacion real: 11 Clasificacion predicha: 11 Porcentaje --> 0.41727653
Clasificacion real: 14 Clasificacion predicha: 9 Porcentaje --> 0.22938408
Clasificacion real: 17 Clasificacion predicha: 17 Porcentaje --> 0.70169353
Clasificacion real: 7 Clasificacion predicha: 7 Porcentaje --> 0.81406707
Clasificacion real: 1 Clasificacion predicha: 12 Porcentaje --> 0.1588758
Clasificacion real: 11 Clasificacion predicha: 11 Porcentaje --> 0.90359914
Clasificacion real: 2 Clasificacion predicha: 2 Porcentaje --> 0.5503853
Clasificacion real: 2 Clasificacion predicha: 9 Porcentaje --> 0.26228827
Clasificacion real: 13 Clasificacion predicha: 3 Porcentaje --> 0.13327564
Clasificacion real: 5 Clasificacion predicha: 5 Porcentaje --> 0.6482591
Clasificacion real: 15 Clasificacion predicha: 4 Porcentaje --> 0.19572468
Clasificacion real: 4 Clasificacion predicha: 4 Porcentaje --> 0.18225403
Clasificacion real: 0 Clasificacion predicha: 10 Porcentaje --> 0.29552302
Clasificacion real: 8 Clasificacion predicha: 9 Porcentaje --> 0.5017741
Clasificacion real: 1 Clasificacion predicha: 1 Porcentaje --> 0.38443804
Clasificacion real: 10 Clasificacion predicha: 10 Porcentaje --> 0.21943092
Porcentaje acierto: 66.66666666666666

```

Figura 31. Predicción con lr fijo

Por otro lado, el modelo con learning rate variable (Figura 32) ha dado unos resultados sustancialmente mejores en cuanto a porcentaje de acierto se refiere, obteniendo un porcentaje del 86.1%, 20% más que el modelo anterior. Este modelo sí que consigue realizar una clasificación consistente acercándose al 90% de acierto.

```

Clasificacion real: 1 Clasificacion predicha: 1 Porcentaje --> 0.99996555
Clasificacion real: 9 Clasificacion predicha: 9 Porcentaje --> 0.999995
Clasificacion real: 2 Clasificacion predicha: 2 Porcentaje --> 0.7401023
Clasificacion real: 3 Clasificacion predicha: 3 Porcentaje --> 0.99934703
Clasificacion real: 5 Clasificacion predicha: 5 Porcentaje --> 0.9999347
Clasificacion real: 2 Clasificacion predicha: 2 Porcentaje --> 0.9999714
Clasificacion real: 1 Clasificacion predicha: 1 Porcentaje --> 0.9995189
Clasificacion real: 3 Clasificacion predicha: 3 Porcentaje --> 0.9999726
Clasificacion real: 11 Clasificacion predicha: 11 Porcentaje --> 0.9999982
Clasificacion real: 15 Clasificacion predicha: 15 Porcentaje --> 0.99992514
Clasificacion real: 15 Clasificacion predicha: 15 Porcentaje --> 0.9998871
Clasificacion real: 9 Clasificacion predicha: 9 Porcentaje --> 0.7069194
Clasificacion real: 1 Clasificacion predicha: 1 Porcentaje --> 0.9999964
Clasificacion real: 4 Clasificacion predicha: 4 Porcentaje --> 0.9999989
Clasificacion real: 9 Clasificacion predicha: 9 Porcentaje --> 0.99989355
Clasificacion real: 9 Clasificacion predicha: 9 Porcentaje --> 0.98982304
Clasificacion real: 13 Clasificacion predicha: 13 Porcentaje --> 0.9999938
Clasificacion real: 7 Clasificacion predicha: 7 Porcentaje --> 0.9999949
Clasificacion real: 17 Clasificacion predicha: 7 Porcentaje --> 0.76878977
Clasificacion real: 2 Clasificacion predicha: 11 Porcentaje --> 0.87277603
Clasificacion real: 11 Clasificacion predicha: 3 Porcentaje --> 0.98974425
Clasificacion real: 0 Clasificacion predicha: 0 Porcentaje --> 0.9438832
Clasificacion real: 9 Clasificacion predicha: 9 Porcentaje --> 0.9999917
Clasificacion real: 7 Clasificacion predicha: 7 Porcentaje --> 0.99927586
Clasificacion real: 7 Clasificacion predicha: 7 Porcentaje --> 0.9350327
Clasificacion real: 16 Clasificacion predicha: 16 Porcentaje --> 0.9999411
Clasificacion real: 5 Clasificacion predicha: 5 Porcentaje --> 0.99991584
Clasificacion real: 14 Clasificacion predicha: 14 Porcentaje --> 0.99997723
Clasificacion real: 16 Clasificacion predicha: 16 Porcentaje --> 0.99994123
Clasificacion real: 6 Clasificacion predicha: 13 Porcentaje --> 0.6992203
Clasificacion real: 9 Clasificacion predicha: 9 Porcentaje --> 0.9998274
Clasificacion real: 9 Clasificacion predicha: 9 Porcentaje --> 0.72255415
Clasificacion real: 9 Clasificacion predicha: 9 Porcentaje --> 0.99785775
Clasificacion real: 15 Clasificacion predicha: 7 Porcentaje --> 0.9841834
Clasificacion real: 6 Clasificacion predicha: 6 Porcentaje --> 0.99925345
Clasificacion real: 3 Clasificacion predicha: 3 Porcentaje --> 1.0
Porcentaje acierto: 86.111111111111

```

Figura 32. Predicción modelo con lr variable

Problemas encontrados

En este apartado se van a exponer los distintos problemas que han ido surgiendo a lo largo de la realización de este TFG y las soluciones que se han llevado a cabo. Durante el desarrollo de este proyecto, se identificaron varios problemas que afectaron significativamente el progreso y la eficacia de éste. A continuación, se detallan los principales problemas encontrados:

- Escasez de imágenes de trufas: La limitada disponibilidad de imágenes de trufas ha sido un obstáculo significativo en el proceso de entrenamiento de la red neuronal, afectando directamente a la capacidad del modelo para generalizar y reconocer trufas en diversas condiciones. Como solución, se ha optado por realizar rotaciones de las imágenes y generar las imágenes en modo espejo para ampliar de esta manera el dataset encargado de entrenar la red neuronal.
- Falta de Redes Neuronales específicas para trufas: La falta de redes neuronales específicamente diseñadas para el reconocimiento de trufas también fue un problema importante. La mayoría de las arquitecturas pre-entrenadas y disponibles públicamente están orientadas hacia conjuntos de datos más generales, lo que limitó la capacidad del modelo para capturar características particulares de las trufas de manera precisa. Se ha tomado la decisión de crear una red neuronal propia.
- Problemas con la máquina utilizada: La máquina utilizada para la implementación del proyecto experimentó inconvenientes técnicos que impactaron negativamente en la continuidad del trabajo. Problemas recurrentes con los controladores (drivers), reinicios inesperados y reinstalaciones de software fueron contratiempos recurrentes que afectaron el desarrollo fluido del proyecto.
- Problemas con Google Colab: En un principio se iba a utilizar Google Colab para realizar la comparativa entre GPU, CPU y Google Colab, pero debido a algunas limitaciones que impone Google y a los tiempos de entrenamiento tan elevados no ha sido posible utilizar esta herramienta como opción de entrenamiento.

Licencia software y documental

La licencia de software que se va a emplear es la denominada Berkeley Software Distribution (BSD). Se trata de una licencia de software libre permisiva como puede ser OpenSSL o la MIT License. Existen diferentes tipos de licencias, en el caso de este TFG se ha utilizado la licencia “BSD modificada”, “BSD revisada”, “BSD-3” o “BSD de 3 cláusulas” [22].



Figura 33. Logotipo de licencia BSD [12]

Por otro lado, este documento ya está amparado por la licencia documental [3] porque forma parte de ZAGUAN, el Repositorio Institucional de Documentos de la Universidad de Zaragoza.

Conclusiones y trabajo futuro

En este trabajo se ha abordado el problema del reconocimiento y clasificación de trufas mediante técnicas de machine learning como paso inicial para la detección de calidad de la trufa, y en concreto redes neuronales convolucionales (CNN). Se ha creado la CNN utilizando el framework de Tensorflow y se ha utilizado un conjunto de datos propio para su entrenamiento. Se ha evaluado el rendimiento de esta red utilizando distintos entornos de ejecución (CPU y GPU)

Tras ello, se ha determinado que las CNN son un método útil a la hora de realizar clasificación de imágenes, y en este caso hacen de punto de partida para posibles nuevos proyectos de detección de calidad en las trufas.

También se ha demostrado que el uso de la GPU es más eficiente tanto en tiempo de entrenamiento como en calidad de entrenamiento, ya que una red neuronal convolucional, ejecutada utilizando GPU, tarda mucho menos tiempo de ejecución y obtiene unos mejores resultados.

Este TFG resuelve el problema planteado para el que ha sido necesario disponer tanto de conocimientos del área de hardware como de software, adquiridos a lo largo de la titulación del Grado en Ingeniería Informática (GII). El trabajo conjunto del datawarehouse de imágenes y redes neuronales ha sido la parte más desafiante a la que ha habido que enfrentarse durante la realización de este trabajo, ya que lo más habitual en redes neuronales es trabajar con información textual o valores numéricos.

En cuanto al trabajo futuro, este TFG puede ser un punto de partida para posibles nuevos trabajos y proyectos que busquen analizar la calidad de las trufas, siendo la ampliación del dataset una apuesta prometedora para realizar un clasificador más consistente. Para finalizar, comentar que se podría también trabajar con otra línea de mejora de interoperabilidad, compatibilidad, etc. [18] de las herramientas utilizadas, por ejemplo, para la configuración de la máquina, en relación con los problemas recurrentes con los controladores (drivers), reinicios inesperados por actualizaciones o caídas del sistema y reinstalación de software.

Referencias bibliográficas

- [1] BARRIO, J., 2008. Redes neuronales para determinar el precio de la vivienda. *Top Webs de Arquitectura y Urbanismo* [en línea]. [consulta: 30 noviembre 2023]. Disponible en: <https://www.urbanismo.com/arquitecturayurbanismo/redes-neuronales-para-determinar-el-precio-de-la-vivienda/>.
- [2] CALVO, D., 2017. Red Neuronal Convolutacional CNN. *Diego Calvo* [en línea]. [consulta: 30 noviembre 2023]. Disponible en: <https://www.diegocalvo.es/red-neuronal-convolutacional/>.
- [3] CC BY-NC-ND 3.0 ES Deed | Atribución-NoComercial-SinDerivadas 3.0 España | Creative Commons. [en línea], [sin fecha]. [consulta: 14 noviembre 2023]. Disponible en: <https://creativecommons.org/licenses/by-nc-nd/3.0/es/>.
- [4] Cultivo de setas silvestres, trufas y micorrizas. *Micofora* [en línea], [sin fecha]. [consulta: 14 noviembre 2023]. Disponible en: <https://micofora.com/>.
- [5] DESINGU, K., BHASKAR, A., PALANIAPPAN, M., CHODISETTY, E.A. y BHARATHI, H., [sin fecha]. Classification of Fungi Species: A Deep Learning Based Image Feature Extraction and Gradient Boosting Ensemble Approach. ,
- [6] Frank Rosenblatt. En: Page Version ID: 153874585, *Wikipedia, la enciclopedia libre* [en línea], 2023. [consulta: 30 noviembre 2023]. Disponible en: https://es.wikipedia.org/w/index.php?title=Frank_Rosenblatt&oldid=153874585.
- [7] Fungipedia. *Fungipedia* [en línea], 2006. [consulta: 14 noviembre 2023]. Disponible en: <https://www.fungipedia.org/>.
- [8] Google Cloud Platform. [en línea], [sin fecha]. [consulta: 14 noviembre 2023]. Disponible en: <https://console.cloud.google.com/>
- [9] Google Colaboratory. [en línea], [sin fecha]. [consulta: 15 noviembre 2023]. Disponible en: <https://colab.research.google.com/drive/1NUWKrIDDaLS-K539TGEsRk9ch28YDGCm?usp=sharing>.
- [10] Hongos hipogeos. [en línea], [sin fecha]. [consulta: 14 noviembre 2023]. Disponible en: <https://www.hongoshipogeos.es/>.
- [11] HUET, P., 2023. Qué son las redes neuronales y sus aplicaciones. *Qué son las redes neuronales y sus aplicaciones* [en línea]. Disponible en: <https://openwebinars.net/blog/que-son-las-redes-neuronales-y-sus-aplicaciones/>.
- [12] Licencia BSD. En: Page Version ID: 154610536, *Wikipedia, la enciclopedia libre* [en línea], 2023. [consulta: 15 noviembre 2023]. Disponible en: https://es.wikipedia.org/w/index.php?title=Licencia_BSD&oldid=154610536.

[13] NA8, 2018. Breve Historia de las Redes Neuronales Artificiales. *Aprende Machine Learning* [en línea]. [consulta: 30 noviembre 2023]. Disponible en: <https://aprendemachinellearning.com/breve-historia-de-las-redes-neuronales-artificiales/>.

[14] Neurona. En: Page Version ID: 155165830, *Wikipedia, la enciclopedia libre* [en línea], 2023. [consulta: 14 noviembre 2023]. Disponible en: <https://es.wikipedia.org/w/index.php?title=Neurona&oldid=155165830>.

[15] Perceptrón. En: Page Version ID: 155349909, *Wikipedia, la enciclopedia libre* [en línea], 2023. [consulta: 30 noviembre 2023]. Disponible en: <https://es.wikipedia.org/w/index.php?title=Perceptr%C3%B3n&oldid=155349909>.

[16] PICEK, L., ŠULC, M., MATAS, J., HEILMANN-CLAUSEN, J., JEPPESEN, T.S. y LIND, E., 2022. Automatic Fungi Recognition: Deep Learning Meets Mycology. *Sensors*, vol. 22, no. 2, ISSN 1424-8220. DOI [10.3390/s22020633](https://doi.org/10.3390/s22020633).

[17] PICEK, L., SULC, M., MATAS, J., JEPPESEN, T.S., HEILMANN-CLAUSEN, J., LASOUE, T. y FROSLEV, T., 2022. Danish Fungi 2020 – Not Just Another Image Recognition Dataset. *2022 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)* [en línea]. Waikoloa, HI, USA: IEEE, pp. 3281-3291. [consulta: 14 noviembre 2023]. ISBN 978-1-66540-915-5. DOI [10.1109/WACV51458.2022.00334](https://doi.org/10.1109/WACV51458.2022.00334). Disponible en: <https://ieeexplore.ieee.org/document/9706991/>.

[18] Portal ISO 25010. *iso25000.com* [en línea], [sin fecha]. [Consulta: 30 noviembre 2023]. Disponible en: <https://iso25000.com/index.php/normas-iso-25000/iso-25010>.

[19] Propagación hacia atrás. En: Page Version ID: 154153117, *Wikipedia, la enciclopedia libre* [en línea], 2023. [consulta: 30 noviembre 2023]. Disponible en: https://es.wikipedia.org/w/index.php?title=Propagaci%C3%B3n_hacia_atr%C3%A1s&oldid=154153117.

[20] SULC, M., PICEK, L., MATAS, J., JEPPESEN, T. y HEILMANN-CLAUSEN, J., 2020. Fungi Recognition: A Practical Use Case. *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision* [en línea]. S.l.: s.n., pp. 2316-2324. [consulta: 14 noviembre 2023]. Disponible en: https://openaccess.thecvf.com/content_WACV_2020/html/Sulc_Fungi_Recognition_A_Practical_Use_Case_WACV_2020_paper.html.

[21] TensorFlow. [en línea], [sin fecha]. [consulta: 14 noviembre 2023]. Disponible en: <https://www.tensorflow.org/?hl=es-419>.

[22] The 3-Clause BSD License. *Open Source Initiative* [en línea], 2011. [consulta: 14 noviembre 2023]. Disponible en: <https://opensource.org/license/bsd-3-clause/>.

[23] Trufamania - trufas comestibles, *Tuber melanosporum*. [en línea], [sin fecha]. [consulta: 14 noviembre 2023]. Disponible en: <https://www.trufamania.com/las-trufas.htm>.

[24] universidad-zaragoza/TRUQU-TRuffle-QUality-Recognition. [en línea], [sin fecha]. [consulta: 29 noviembre 2023]. Disponible en: <https://github.com/universidad-zaragoza/TRUQU-TRuffle-QUality-Recognition>.

[25] Vision AI. *Google Cloud* [en línea], [sin fecha]. [consulta: 14 noviembre 2023]. Disponible en: <https://cloud.google.com/vision>, <https://cloud.google.com/vision?hl=es>.